Distillate: A Meta-Agent Harness for Autonomous ML Research

Romain Lacombe
romain.lacombe@gmail.com

Abstract

Code generation agents now run unsupervised for hours or days, and their value to ML research turns on a measurable property: whether an agent can forecast the outcomes of its own experiments before running them. We introduce **Distillate**, a multi-agent harness for autonomous ML research that addresses three coupled challenges: (1) coherence loss across long-horizon programs; (2) uncalibrated and irreproducible outcome predictions; and (3) thin human-in-the-loop oversight. Distillate combines Recursive Language Model (RLM) agents [Zhang et al., 2025] reasoning over typed Python namespaces in place of serialized prompts, pre-registration enforced as a tool contract with a cryptographic Git anchor, and a conversation-agent Orchestrator that translates researcher intent into a steerable research program. Across nine campaigns and n=326 pre-registered runs over three scientific domains (combinatorial mathematics, computer vision, molecular biology), pre-registered predicted improvement $\Delta\hat{y}$ tracks realized Δy at Pearson r=+0.35 pooled (p<0.001); within-campaign effects range from r=+0.59 on GlycoGen to r=+0.28 (n.s.) on CIFAR-100 [Krizhevsky, 2009] — a modest, domain-heterogeneous signal rather than uniform calibration. Self-rated confidence c is model-specific — the sign of vendor-level correlations depends on metric choice (raw vs. signed) and both formulations give pooled correlations near zero — so a single portable confidence scalar is not deliverable today. As one concrete case, a Distillate Experimentalist reached state-of-the-art on de novo glycan structure prediction (companion paper). Our hope is that harnesses like Distillate augment research rather than automate it, giving researchers command over larger scopes, harder questions, and more ambitious programs.

1 Introduction

Are modern CLI agents autonomous ML researchers? Claude Code, Codex, and Gemini CLI now run unsupervised for hours or days, and a growing flurry of agent-driven systems has produced novel algorithms [Romera-Paredes et al., 2024, Novikov et al., 2025, Trinh et al., 2024], chemistry campaigns [Boiko et al., 2023, Bran et al., 2024], biomolecule designs [Swanson et al., 2024], and multi-hour discovery cycles [Lu et al., 2024, Yamada et al., 2025, Edison Scientific, 2025]. These outputs are impressive; the *process* by which agents reach them remains largely opaque. Answering the question rigorously requires a harness that makes that process measurable. Three challenges decide whether autonomous research agents earn the compute they consume or waste it.

(i) Coherence. A research program spans hundreds of papers, dozens of experiments, and thousands of run records. Even recent context-window extensions and per-turn summarization compress this history lossily and asymmetrically: the model cannot programmatically re-inspect the parts it needs, only the parts a prior summarizer chose to preserve. The agent must reason *over* the workspace, not carry it in the prompt, or it loses focus and drifts off-program.

Working paper.

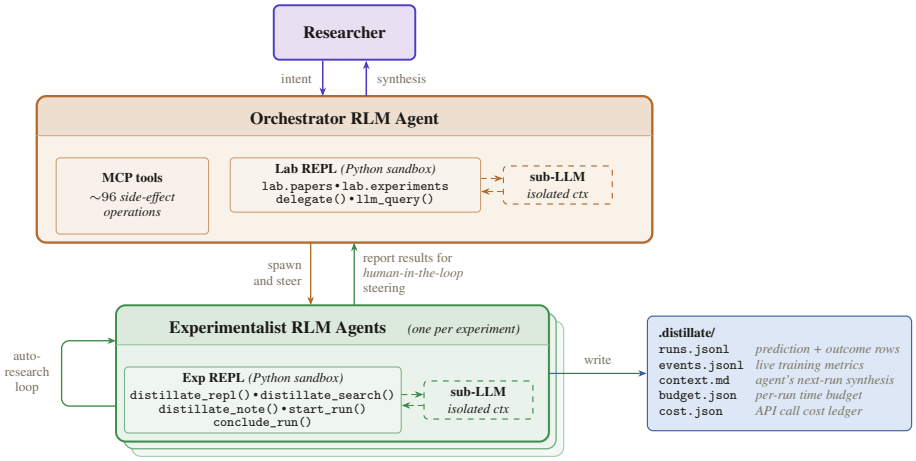


Figure 1: Distillate architecture. The researcher addresses *intent* to the Orchestrator RLM Agent, which evaluates a typed Python facade in the Lab REPL and spawns isolated sub-LLM calls via `delegate()` and `llm_query()`. The Orchestrator launches an Experimentalist RLM Agent (subprocess running without supervision) and disengages; the Experimentalist hill-climbs in the auto-research loop (Figure 2) and writes to `.distillate/`, which the Orchestrator reads back through the Lab REPL when next consulted.

(ii) Calibration and reproducibility. An unsupervised agent commits to no prior expectation, so any outcome can be rationalized post-hoc as what we call *agentic p-hacking*: the agent runs the post-hoc selection pipeline that pre-registration was invented to prevent. Without a content-addressed prediction recorded *before* the run, the trace cannot be reproduced or verified.

(iii) Human-in-the-loop steerability. Research is iterative: each finding reshapes the next question. Agents must work *for* the researcher, redirectable mid-flight when findings warrant a pivot, and aligned with the evolving question rather than the one originally asked.

This paper's central claim: **modern CLI agents produce usable self-forecasts about their own ML research runs on structured optimization tasks**, and this becomes measurable and actionable when they are paired with a harness that resolves the three challenges above. We deliver such a harness through **Distillate**, a principled multi-agent autonomous research steering and orchestration platform that addresses all three (Figures 1, 2). An **Orchestrator** RLM Agent acts as a co-scientist in a chat surface, surfacing literature, comparing past runs, and designing the research program with the researcher. It launches a swarm of autonomous **Experimentalist** RLM Agents (one per experiment) that execute the program without supervision, communicating with the Orchestrator through a content-addressed filesystem (`.distillate/`) and tool contracts (`start_run`, `conclude_run`) that gate training launches on a cryptographically committed pre-registration.

Contributions. Three architectural commitments answer the three challenges, plus one empirical contribution.

(1) Recursive Language Model agents over a research workspace (addresses coherence, Section 3). Both the Orchestrator and the Experimentalists are RLMs [Zhang et al., 2025]: they reason *over* a typed Python workspace (`lab.*` for the Orchestrator, `xp.*` for the Experimentalist) programmatically, rather than serializing the workspace into the prompt. The agent's state is bound as native Python variables it queries on demand, with delegated work spawned to sub-LLM agents in isolated contexts.

(2) Cryptographic pre-registration linking intent to outcome (addresses calibration and reproducibility, Section 3). A tool contract gates training: before any script may launch, the Experimentalist calls `start_run` with a falsifiable numeric prediction and self-rated confidence, pushed to a Git history that traces the experimental path. A two-commit protocol content-addresses the prediction in the run's witness ancestry, making the trace forensically verifiable and foreclosing in-context p-hacking [Lakens, 2022].

(3) Human-in-the-loop Orchestrator + Experimentalist swarm (addresses steerability and alignment, Section 3). The Orchestrator translates the researcher's intent into autonomous runs and surrenders the workspace until next consulted; a bidirectional steering channel (Figure 1, upward arrow) gives the researcher a low-overhead path to redirect a stalled Experimentalist mid-flight without restarting the run.

(4) Empirical (Section 5). To our knowledge, we provide the first systematic measurement of whether autonomous coding agents' pre-registered predictions correlate with realized task success. Across three scientific domains — combinatorial mathematics (Tiny-MatMul), molecular biology (GlycoGen), computer vision (CIFAR-100) — and three frontier coding agents, pre-registered improvement $\Delta\hat{y}$ predicts realized improvement Δy at Pearson $r = +0.35$ (cluster-robust 95% CI $[+0.23, +0.49]$, $p < 0.001$, $n = 326$). As one concrete illustration, the **GlycoGen Experimentalist reached state-of-the-art on de novo glycan structure prediction** [Lacombe, 2026]. However, per-model ECE varies from 0.17 to 0.74 across cells (Section 5) and self-rated confidence is not uniformly calibrated across vendors.

2 Related Work

Recursive language models, tool use, and language-model programs. The term *Recursive Language Model* is due to Zhang et al. [2025], who treat a long prompt as data within a Python REPL and let the root model programmatically inspect, decompose, and recursively spawn sub-LLM calls over slices — an approach that processes inputs roughly two orders of magnitude beyond the model's nominal context window. The companion *Mismanaged Geniuses Hypothesis* [Zhang et al., 2026] argues that frontier LMs are underutilized because the systems managing them rely on brittle human-engineered decomposition. Distillate takes that prescription seriously: both the Orchestrator and the Experimentalists are RLMs over typed Python namespaces (Section 3), so LM management is itself programmatic, principled, and auditable. This work sits at the end of an arc of tool-augmented and program-shaped LM use — **ReAct** [Yao et al., 2023b], **Toolformer** [Schick et al., 2023], **Tree of Thoughts** [Yao et al., 2023a], **Self-Refine** [Madaan et al., 2023], **HuggingGPT** [Shen et al., 2023], **Gorilla** [Patil et al., 2023], **DSPy** [Khattab et al., 2024] — none of which addresses the case where the program persists across sessions and operates over a workspace exceeding any single context window.

Long-horizon agents and autonomous research. Long-horizon agent behavior is evaluated by benchmarks (**GAIA** [Mialon et al., 2023], **AgentBench** [Liu et al., 2024], **WebArena** [Zhou et al., 2024], **RE-Bench** [Wijk et al., 2024], **SWE-bench** [Jimenez et al., 2024, Cognition AI, 2024, Deng et al., 2025], **MLE-bench** [Chan et al., 2024]) and addressed by multi-agent frameworks such as **Magentic-One** [Fourney et al., 2024] and **AutoGen** [Wu et al., 2023]. Applied to science: **ChemCrow** [Bran et al., 2024], **Coscientist** [Boiko et al., 2023], **Virtual Lab** [Swanson et al., 2024, Gao et al., 2024], Sakana's **AI Scientist** [Lu et al., 2024, Yamada et al., 2025, Beel et al., 2025], **Agent Laboratory** [Schmidgall et al., 2025], **Kosmos** [Edison Scientific, 2025], **AIDE** [Jiang et al., 2025], **MLR-Copilot** [Li et al., 2024], **POPPER** [Huang et al., 2025], **FunSearch** [Romera-Paredes et al., 2024], **AlphaEvolve** [Novikov et al., 2025]. A public auto-research culture (Karpathy's open projects [Karpathy, 2022–2025], the **AdderBoard** [Lee et al., 2023, Papailiopoulos and contributors, 2024]) conducts small fully-traced experiments in the spirit Distillate aims to make routine. *None of these systems measure whether the agent's numerical beliefs about its own upcoming actions are calibrated to outcomes.* The Kosmos authors themselves identify the absence of "calibrated confidence levels per claim" as future work. We treat the human as the source of intent, the agent as a measurable epistemic actor, and pre-registration as the instrument that makes its forecasts falsifiable.

Self-improving agents, durable memory, and orchestration protocols. **Reflexion** [Shinn et al., 2023] introduced verbal reinforcement learning; **Voyager** [Wang et al., 2023] an open-ended skill library; **Generative Agents** [Park et al., 2023] reflective memory streams; **MemGPT** [Packer et al., 2023] formalizes hierarchical memory. All share the conviction that productive autonomy at extended horizons requires durable, agent-authored memory; all keep that memory internal to the agent. Distillate externalizes it to a filesystem protocol that survives process boundaries, permitting an experiment to be paused, resumed under a different harness, and inspected by both user and shell. Hierarchical orchestration is now standard [Wu et al., 2023] (LangGraph, CrewAI, AutoGen [Wu et al., 2023]); the **Model Context Protocol** [Anthropic, 2024] makes Distillate's Experimentalist

backends interchangeable. We depart from the orchestration mainstream in insisting that long-running subprocess agents *not* be supervised during execution, since polling is hostile to multi-day timescales.

LLM confidence and calibration in scientific tasks. LLMs frequently know more than they verbalize about their own uncertainty [Kadavath et al., 2022, Lin et al., 2022], yet remain miscalibrated under realistic prompts: **ClimateX** [Lacombe et al., 2023] finds systematic overconfidence on expert-graded scientific statements; **Don't Think Twice** [Lacombe et al., 2025] shows that extended reasoning budgets *worsen* calibration on reasoning LLMs, a counterpoint to test-time-scaling. Both measure calibration on fixed text instances against external ground truth. The pre-registration substrate lifts the question to *agent self-forecasting under autonomous execution*, where the prediction concerns a training run the agent is about to conduct: a new setting with higher outcome uncertainty, no fixed ground truth, and the additional complication that the agent's own choices determine the outcome it is predicting (Section 5).

Pre-registration. Originating in the social and clinical sciences as a remedy for p-hacking and HARK-ing [Lakens, 2022], pre-registration was adapted to predictive modeling by Hofman et al. [2023] in the context of an ML reproducibility crisis [Kapoor and Narayanan, 2023, Semmelrock et al., 2025, Huntington-Klein et al., 2025]. The literature argues for it as a discipline; Distillate enforces it via the structure of the agent's tool surface (Section 3). None of these prior works articulates pre-registration as a tool-contract problem.

3 The Distillate Harness

Distillate's two-agent architecture (Figure 1) realizes the three architectural commitments at the system level. The **Orchestrator** (named to the user as Nicolas, after the alchemist Nicolas Flamel [Ball, 2025]) is a persistent in-process agent on the Claude Agent SDK with $\sim$96 MCP tools spanning papers, experiments, projects, sessions, and runs; it acts as a co-scientist in a chat surface, surfacing literature, comparing past runs, and launching or steering experiments. **Experimentalists** are `claude` sessions running in `tmux`, one per experiment, that hill-climb against an objective metric without supervision. The architecture is harness-agnostic by design (MCP tool contract, filesystem-only state).

3.1 RLM agents over a research workspace (coherence)

Both agents run inside a sandboxed Python REPL with a typed namespace (`lab.*` for the Orchestrator, `xp.*` for the Experimentalist) restricted to the workspace API (no `import`/`eval`/`exec`). The workspace is bound as native Python variables the agent queries on demand: methods such as `lab.experiments.recent(5).runs.peak_metric()` return fluent collections in-process, in the spirit of Recursive Language Models [Zhang et al., 2025]. Sub-LLM work follows a cost ladder from \$0 (direct namespace access) through `llm_query` (one Haiku call) to `delegate` (a full Claude Code subprocess, up to a flagship model); a per-experiment ledger (`.distillate/cost.json`) is atomically updated on every sub-LM call so the agent can self-budget across the multi-day program.

3.2 Pre-registration as a cryptographic intent-to-outcome link (calibration, reproducibility)

Before any training script may launch, the Experimentalist must call `start_run` with a seven-field schema (*description, hypothesis, prediction, predicted_metric, predicted_value, confidence in* $[0, 100]$*, rationale*; full schema in Appendix A), and the `run_id` it returns is the only valid identifier for any subsequent training-related call. The schema records the agent's epistemic state at every run: *what* it believes it is trying, *why* it expects it to work, *how much* improvement it anticipates, and *how confident* it is. After training, `conclude_run` writes the actual results, an agent-authored verdict (`confirmed` / `refuted` / `inconclusive`, a subjective self-assessment), and a `belief_update` field. Downstream calibration analysis uses a separate objective *analysis verdict* $v_n = \mathbb{1}[|y_n - \hat{y}_n| \leq \tau]$ with $\tau = 0.10$ (Section 5). Both calls write to an append-only `runs.jsonl`, with each pre-registration row paired to its conclusion row by a shared `run_id`. We implement pre-registration [Lakens, 2022, Hofman et al., 2023] structurally rather than as a discipline. The longitudinal trace of agent beliefs paired with verifiable outcomes is the substrate for the calibration analyses in Section 5.

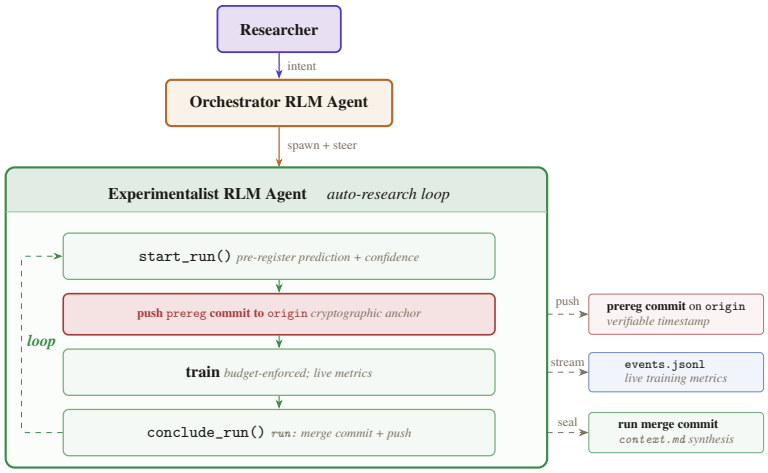


Figure 2: The auto-research loop inside one Experimentalist run. Each iteration runs the four-step cycle `start_run` → `git push prereg` → `train` → `conclude_run`; the brace marks the tool-gated pre-registration contract enforced by the MCP server (the agent cannot launch training until the prereg commit is on `origin`). Each step writes to a specific artifact in `.distillate/`; the next iteration reads the prior `context.md` at launch.

The pre-registration commit is anchored on `origin` as *one run = two Git commits* (Figure 2). The `prereg:` commit stages the prereg row and is pushed *before* training launches; the `run:` commit, emitted by `conclude_run`, is a merge commit whose ancestry incorporates anything the Orchestrator pushed during training. Once the prereg is pushed, the prediction cannot be modified without breaking the commit hash chain, the remote stamps an arrival time outside the agent's control, and a third-party reader can walk the history to verify exactly what evidence the agent saw before each run and that no prediction was silently rewritten. Each experiment is its own Git repository with a `.distillate/` directory (Figure 1, sidebar) holding the agent-readable state; the filesystem plus Git history is the entire inter-agent communication channel. For each run n the pre-registration tuple is $P_n = (m_n, \hat{y}_n, c_n)$ with m_n the predicted metric, $\hat{y}_n \in \mathbb{R}$ the predicted value, $c_n \in \{0, \ldots, 100\}$ the self-rated confidence; the analysis dataset is $\mathcal{D} = \{(P_n, y_n, v_n)\}_{n=1}^{N}$ where y_n is the observed value of m_n and v_n is the objective analysis verdict defined above, agent- or tool-authored end-to-end with no manual labelling step.

3.3 Human-in-the-loop Orchestrator and Experimentalist swarm (steerability, alignment)

Mainstream multi-agent orchestration [Wu et al., 2023] polls workers from the orchestrator, a pattern hostile to multi-day timescales. Distillate disengages instead: the Orchestrator launches the Experimentalist and surrenders the workspace until next consulted. The pre-registration trace then makes the trigger condition for human intervention well-defined — consecutive prereg/outcome rows with no belief update — and the Orchestrator's upward channel (Figure 1) lets the researcher inject a single corrective hint on the next `start_run` without restarting the session. Mainstream coding agents already support mid-flight user redirection; the novelty here is not in the redirection mechanism itself but in the *trigger* being made legible from the pre-registration trace — an Orchestrator (or human) scanning `runs.jsonl` sees the moment an agent's predictions and outcomes diverge with no belief update, which is exactly the intervention point. The Git-tracked `.distillate/` provides the audit trail that supports this loop by construction (Section 5 reports the corresponding Gemini Flash unstuck-by-one-hint instance).

Reproducibility. Distillate is released as an open-source Python package; the harness code, the MCP tool contracts, the pre-registration `runs.jsonl` traces for every experiment in Section 4, and the analysis script that reproduces every number in Tables 1–3 are all publicly available at `https://github.com/rlacombe/distillate`. Orchestrator and Experimentalist system prompts are excerpted in Appendix B.

4 Experiments

To test whether autonomous research agents can be trusted with multi-day work, we deploy Distillate across three experimental domains chosen to span scientific surfaces with different ground truths: *combinatorial mathematics* (a closed analytic problem where the agent satisfices under tight compute budget), *molecular biology* (an open scientific problem with no analytic answer, only experimental ground truth), and *computer vision* (a canonical benchmark where the agent hill-climbs an accuracy ceiling). All are driven through Distillate and pre-registered run-by-run. Tiny-MatMul and CIFAR-100 are run across all three coding agents (Claude Code, Codex CLI, Gemini CLI); GlycoGen is a longer single-agent campaign on Claude Code, used to push depth on a single research program rather than breadth across models.

Combinatorial mathematics: Tiny-MatMul. Smallest transformer that solves $N \times N$ mod-10 matrix multiplication under a 5-min wall-clock training budget; the agent satisfices on accuracy (≥ 0.99) while minimizing parameter count. The 5×5 deployment is single-agent (Sonnet 4.6 on Claude Code, Apple Silicon MPS); the 3×3 deployment is three-model on Sonnet-class agents (Sonnet 4.6 on Claude Code, GPT-5.4-mini on Codex CLI, Gemini 3 Flash on Gemini CLI), chosen for budget reasons given the long horizons of multi-model campaigns ($\sim$6 h of agent wall-clock per task at default reasoning, compounded across two task families). Pooled across all four deployments, $n_{\mathrm{mm}} = 203$ on-metric pairs.

Molecular biology: GlycoGen. De novo glycan structure prediction from tandem mass spectrometry, under the discrete flow matching family. Single-agent campaign (Sonnet 4.6 on Claude Code) deployed at two rungs of the scaling ladder: 80 runs on Apple Silicon (MPS, $\sim$23 h active compute) and 21 runs on an A100 GPU (101 runs total). Primary metric throughout is `top1_structural_accuracy`; after pairing pre-registrations to `conclude_run` rows and excluding runs with mid-experiment metric switches or incomplete conclusion, $n_{\mathrm{glyco}} = 49$ on-metric pairs. **The Experimentalist's autonomous run reached state-of-the-art on de novo glycan structure prediction**; the architecture and ablations are verified independently in a companion paper [Lacombe, 2026], demonstrating that autonomous agents can conduct ML research at the frontier of a scientific discipline.

Computer vision: CIFAR-100 [Krizhevsky, 2009]. Three-model experiment on dedicated NVIDIA H200 GPUs via Hugging Face Jobs — the same three frontier agents as Tiny-MatMul 3×3 but on research-grade GPUs, accuracy-maximization protocol, $\leq$ 5M-parameter classifier cap, 5-min per-run and 6-hour total training budget, fixed 45K/5K train/val split with the 10K test set held out. Agents hill-climb on *val accuracy*; the held-out test set is not visible during the campaign. Reported per-run $\hat{y}$ and y in Section 5 are val-accuracy values (the target the agent is optimizing and forecasting); a single held-out-test number for the best-per-vendor model is out of scope here and left to future work. The campaign provides $n_{\mathrm{cifar}} = 74$ on-metric pairs.

The combined on-metric dataset $\mathcal{D}$ has $n = 326$ prediction–outcome pairs across $k = 9$ sequential campaigns.

Analysis formalism. We compute deltas relative to the running maximum within each campaign:

$$\Delta\hat{y}_n = \hat{y}_n - \max_{k<n} y_k, \qquad \Delta y_n = y_n - \max_{k<n} y_k. \tag{1}$$

The deployment question *"how much will this run push the frontier?"* reduces to the magnitude correlation $r(\Delta\hat{y}, \Delta y)$ on the pooled Δ-pairs. Deltas absorb two confounds that inflate raw predicted-vs-observed correlations: a within-campaign time trend on accuracy-maximization tasks, and a saturation artifact on satisficing tasks (most predictions cluster near the gate). Significance and confidence intervals come from a cluster-robust bootstrap that resamples whole campaigns (B=5,000 resamples, fixed seed), since within-campaign runs are sequentially correlated. Reported p-values and CIs are nominal and uncorrected across per-experiment / per-model cells — to be revisited under FDR control as the dataset grows.

The self-rated confidence channel is summarized by two quantities: the correlation $r(c, \Delta y)$ on the delta-pairs (raw r_2; tabulated in Tables 1 and 2 for continuity with prior reporting) and its signed alternative $r_2^\star = r(c, \mathrm{sign}(\Delta\hat{y}) \cdot \Delta y)$ (Table 3), which removes a confound the raw form carries on decline predictions: a confident, correct negative prediction contributes to the raw r_2 as anti-correlation rather than as a calibration success, whereas $r_2^\star$ credits it as a directional hit; and, on the

verdict dataset, the standard binned Expected Calibration Error

$$\mathrm{ECE} = \sum_{b=1}^{B} \frac{|\mathcal{B}_b|}{N} \left|\mathrm{acc}(\mathcal{B}_b) - \mathrm{conf}(\mathcal{B}_b)\right|, \tag{2}$$

with confidence bins $\mathcal{B}_b$ of width 10 over $[0, 100]$.

5 Results

To our knowledge, this is the first systematic measurement of whether autonomous coding agents' pre-registered predictions correlate with realized task success on multi-day research campaigns. Prior work measures calibration on fixed text instances against external ground truth [Lacombe et al., 2023, 2025]; here we lift the question to agent self-forecasting under autonomous execution, where the prediction concerns a training run the agent is about to conduct, with no fixed ground truth and the additional complication that the agent's own choices determine the outcome. Two findings stand out: $\Delta\hat{y}$ carries deployable signal across all three models, while the self-rated-confidence channel *c flips sign* between them — breaking the assumption that LLM self-confidence is a portable scalar.

Headline: predicted improvement tracks realized improvement. Pooled across nine campaigns and n=326 runs spanning three scientific domains and three frontier coding agents, pre-registered improvement $\Delta\hat{y}$ tracks realized improvement Δy at Pearson r=+0.35 (cluster-robust 95% CI $[+0.23, +0.49]$, p<0.001). Autonomous in-context descent beats random; the agent's own numerical forecast about a run it has not yet executed carries deployable signal. Within-campaign effect sizes range from r=+0.59 (GlycoGen, n=49) to r=+0.28 (CIFAR-100 / Sonnet, n=28); the pooled estimate absorbs this heterogeneity rather than implying uniform calibration across domains. We report four signals per cell: the magnitude correlation $r_1 = r(\Delta\hat{y}, \Delta y)$, the confidence-to-improvement correlation $r_2 = r(c, \Delta y)$, the frontier-push hit rate $P(\Delta y{>}0 \mid \Delta\hat{y}{>}0)$ on the frontier-push subset $\mathcal{F}$, and the binned ECE under the verdict $|y - \hat{y}| \le 0.10$.

Within-Claude: Sonnet 4.6 across the three experiments. Table 1 reports four signals per experiment for the within-Claude depth-first deployments: the magnitude correlation $\Delta\hat{y}{\to}\Delta y$, the confidence-to-improvement correlation $c{\to}\Delta y$, the frontier-push hit rate $P(\Delta y{>}0 \mid \Delta\hat{y}{>}0)$, and binned ECE under the verdict confirmed $\iff |y - \hat{y}| \le 0.10$. The magnitude signal is significant on GlycoGen and Tiny-MatMul ($r{=}+0.59$, $p{<}10^{-6}$ and $r{=}+0.54$, $p{<}10^{-6}$); on CIFAR-100 the signal is positive but underpowered (n=28, p=0.15). The confidence channel is significant only on Tiny-MatMul ($r{=}+0.40$, $p{<}10^{-4}$). ECE is lowest on Tiny-MatMul (where Sonnet's predictions and outcomes both saturate near a clear gate) and highest on CIFAR-100 (small n, mostly Sonnet was conservative).

Table 1: Within-Claude (Sonnet 4.6 / Claude Code) calibration signals across the three experiments. $r_1 = \Delta\hat{y}{\to}\Delta y$ Pearson; $r_2 = c{\to}\Delta y$ Pearson; hit rate is $P(\Delta y{>}0 \mid \Delta\hat{y}{>}0)$ on the frontier-push subset of size $|\mathcal{F}|$; ECE binned at width 10 over $[0, 100]$ with verdict $|y - \hat{y}| \le 0.10$.

Experiment	n	r_1	p	r_2	p	$\lvert\mathcal{F}\rvert$	hit	ECE
GlycoGen	49	+0.59	$< 10^{-6}$	+0.24	0.10	41	49%	0.36
Tiny-MatMul	69	+0.54	$< 10^{-6}$	+0.40	$< 10^{-4}$	12	67%	0.19
CIFAR-100*	28	+0.28	0.15	+0.18	0.35	28	22%	0.52

*Sonnet slice of the cross-model H200 deployment; see Table 2 for the full cross-model result.

Cross-model: Claude Code Sonnet 4.6 vs. Codex GPT-5.4-mini vs. Gemini 3 Flash. The magnitude signal $\Delta\hat{y}{\to}\Delta y$ replicates beyond Claude Code on Tiny-MatMul and is directionally consistent on CIFAR-100 (Table 2; per-cell power limited at the smaller-model tier). The *confidence* channel $c{\to}\Delta y$ varies in sign across vendors under the raw formulation, but that pattern is partly a metric artefact (Table 3 and the paragraph below); the pooled signal is near zero under either metric, and the vendor-level sign pattern reverses when we correct for the fact that confidence attaches to a signed prediction. The substantive finding is that the confidence channel is per-model rather than portable, regardless of formulation. Per-model ECE on Tiny-MatMul follows the same ordering as the predicted-magnitude trust signal.

Cross-vendor: best model produced. The campaigns also benchmark *what each agent built*. On Tiny-MatMul 3×3 (smallest passing transformer at $\geq 99\%$ accuracy), the best architecture is Claude Code Sonnet 4.6's 400 params at 99.2%, Codex GPT-5.4-mini's $1{,}170$ at 99.6%, Gemini 3 Flash's $2{,}046$ at 99.0% — a $\sim5\times$ spread, Claude Code Sonnet 4.6 smallest. On CIFAR-100 ($\leq$5M params, 5-min H200 cap), peak val accuracy is Claude Code Sonnet 4.6 77.6%, Codex GPT-5.4-mini 74.0%, Gemini 3 Flash 72.4%. On Tiny-MatMul the best-calibrated model (lowest ECE, Sonnet 4.6) is also the model that built the smallest passing transformer; on CIFAR-100 the ranking splits (Sonnet produces the best final model but Gemini has the lowest ECE), so calibration quality and produced-model quality are not monotonically aligned across settings. Together, these cross-vendor outcomes and per-model calibration signals form a natural substrate for a long-horizon research-task benchmark; a formal leaderboard with more models, tasks, and pairwise outcomes is left to future work.

Table 2: Per-model breakdown on the two cross-model experiments. Same column conventions as Table 1. The $\Delta\hat{y}\to\Delta y$ channel replicates across models; the $c\to\Delta y$ channel is model-specific in sign and significance.

Experiment	Model	n	r_1	p	r_2	p	$\lvert\mathcal{F}\rvert$	hit	ECE
Tiny-MatMul	Sonnet 4.6	69	$+0.54$	$< 10^{-6}$	$+0.40$	$< 10^{-4}$	12	67%	0.19
	GPT-5.4-mini	94	$+0.39$	$< 10^{-4}$	-0.28	0.006	3	33%	0.28
	Gemini 3 Flash	40	$+0.38$	0.02	-0.30	0.06	18	22%	0.30
CIFAR-100*	Sonnet 4.6	28	$+0.28$	0.15	$+0.18$	0.35	28	22%	0.52
	GPT-5.4-mini	33	$+0.23$	0.19	$+0.05$	0.78	33	33%	0.74
	Gemini 3 Flash	13	$+0.21$	0.49	$+0.26$	0.39	13	–	0.17

*Cross-model CIFAR-100 deployment on H200 GPUs via Hugging Face Jobs.

Signed r_2 correction and baselines. The raw $r_2 = r(c, \Delta y)$ reported in Tables 1–2 is confounded when the agent frequently predicts declines (small $|\mathcal{F}|/n$, as with Codex on Tiny-MatMul, $|\mathcal{F}|$=3 of 94): a confident, correct decline ($\Delta\hat{y}<0$, c high, $\Delta y<0$) contributes as anti-correlation rather than as a calibration success. The signed alternative $r_2^\star = r(c, \operatorname{sign}(\Delta\hat{y})\cdot\Delta y)$ removes this confound. Recomputed on the accessible campaign subset (n=288 of 326, 8 of 9 campaigns; GlycoGen v1 archival files were unavailable), the sign of every vendor's r_2 *reverses* between the two formulations (Table 3) — the vendor sign-flip pattern the raw metric shows is thus a metric artefact, not a durable model-family property. Both formulations, however, yield pooled correlations near zero (raw -0.05, signed $+0.10$, both n.s.), so the substantive conclusion is robust: the confidence channel is per-model, not portable, whichever metric is used.

Baselines. Against the pooled agent $\Delta\hat{y}\to\Delta y$ of r=$+0.36$ on this subset, three null baselines: a within-campaign permutation of $\Delta\hat{y}$ (B=5,000 shuffles) gives r=$+0.02$ (95% $[-0.11, +0.14]$); a frozen-first-prediction (re-use the first run's $\hat{y}$ for every subsequent $\Delta\hat{y}$) gives r=$+0.16$; a constant-mean predictor gives r=0 by construction. Agent updating contributes substantially over the static-guess baseline, and r=$+0.36$ sits well above the permutation null.

Table 3: Raw vs. signed confidence-channel statistic, pooled per vendor across all their experiments (accessible subset n=288). Signed $r_2^\star = r(c, \operatorname{sign}(\Delta\hat{y})\cdot\Delta y)$ removes the raw metric's confound on decline predictions. The vendor sign pattern reverses between the two formulations; both yield pooled correlations near zero.

Model	n	raw r_2	signed $r_2^\star$	difference
Sonnet 4.6	108	$+0.38$	-0.18	-0.56
GPT-5.4-mini	127	-0.38	$+0.27$	$+0.65$
Gemini 3 Flash	53	-0.25	$+0.35$	$+0.60$
Pooled	288	-0.05	$+0.10$	$+0.15$

Human-in-the-loop steering unlocks attractors. Gemini 3 Flash on Tiny-MatMul 3×3 stalled across ten autonomous runs from two compounding causes (architectures too small to clear the gate; misread training-budget instructions) — both visible in `runs.jsonl` as predicted accuracy near 0.95 against an actual parity floor at ~0.20, yet the agent did not update its preregs. A corrective hint through Orchestrator steering broke the attractor: the next run trained a $10{,}200$-parameter transformer

for the full duration and hit 99% accuracy. The pre-registered trace makes intervention points *legible* — a researcher (or the Orchestrator on their behalf) scanning `runs.jsonl` sees the moment an agent stalls and can inject a corrective hint without restarting the run.

Case study: glycan structure prediction. On the GlycoGen application [Lacombe, 2026], the Distillate loop explored 21 pre-registered hyperparameter configurations across five architectural dimensions, converging on a 50M-parameter discrete flow matching model that **exceeded the prior state-of-the-art on de novo glycan structure prediction under a leak-free per-spectrum protocol**: 0.9251 top-1 structural accuracy at $K{=}8$ sampling versus 0.8092 top-1 for GlycoBART [Abtheen et al., 2025] evaluated under the same protocol (Lacombe 2026 contains the full comparison; the originally published GlycoBART number of 0.891 uses a leaky spectrum-level split with database snap-back and sample-level aggregation that is not available in single-spectrum streaming deployment). The autonomous loop also surfaced an empirical scaling regularity (training-step count dominates parameter count for sample efficiency) — concrete evidence that RLM-driven autonomous ML research is deliverable when the search space is structured and the metric unambiguous.

6 Limitations and Future Work

Calibration scope and confidence metric. The deepest within-model campaign (GlycoGen) uses Sonnet 4.6 / Claude Code only; cross-model experiments use Sonnet-class agents under a 5-min per-run budget. The pooled $r{=}{+}0.35$ absorbs domain heterogeneity (per-experiment r from $+0.59$ on GlycoGen to $+0.28$ on CIFAR-100) — a population-average, not a within-domain claim. The confidence-channel finding is metric- dependent: raw $r_2 = r(c, \Delta y)$ (Tables 1 and 2) is confounded on decline predictions, and the signed alternative $r_2^\star = r(c,\, \mathrm{sign}(\Delta\hat{y}){\cdot}\Delta y)$ reverses the vendor pattern (Table 3); both yield pooled values near zero. The recomputation is on the accessible campaign subset ($n{=}288$ of 326, 8 of 9 campaigns); GlycoGen v1 archival files were unavailable at revision time. Fully re-cutting Tables 1 and 2 under $r_2^\star$ once v1 is restored is future work.

Reproducibility and steerability. Pre-registration is enforced by tool contract, not sandboxing: the two-commit Git anchor attests ordering and immutability, but an adversarial agent writing `runs.jsonl` directly bypasses the contract. A sandboxed filesystem mediator would close that gap without changing the user-facing protocol. The Tiny-MatMul $3{\times}3$ deployment required one human-in-the-loop intervention to unstick Gemini Flash from a mod-2 attractor; a fully-autonomous-only ablation with no Orchestrator hint permitted remains open.

Component ablations and benchmark contamination. Isolating each mechanism (typed workspace vs. serialized prompts, pre-registration on/off, Orchestrator on/off) is the largest open experimental gap in this work; the per-model breakdowns in Table 2 act only as a partial natural ablation across the model$\times$harness interface. Contamination is the frontier problem for autonomous-agent evaluation: CIFAR-100 [Krizhevsky, 2009] and modular-arithmetic transformers are widely represented in pretraining corpora, and recall can partly substitute for search. Two design choices bound this concern here — CIFAR-100's $\leq$ 5M-parameter and 5-min compute caps force search beyond direct SOTA recall, and Tiny-MatMul's smallest-passing target has no canonical answer at this scale — while the GlycoGen campaign is a novel de novo structure prediction task unlikely to be memorized. Purpose-built, community-curated, rotating benchmarks are the durable answer.

7 Conclusion

Distillate addresses three coupled challenges — coherence, calibration and reproducibility, steerability — for autonomous agent research. Across $n{=}326$ pre-registered runs over nine campaigns, predicted improvements track realized improvements at $r{=}{+}0.35$ pooled ($p{<}0.001$; per-domain $r{=}{+}0.59$ GlycoGen $\rightarrow$ $r{=}{+}0.28$ n.s. CIFAR-100); self-rated confidence is model-specific; and one Distillate Experimentalist reached SOTA on de novo glycan structure prediction: a modest, honest yes on structured optimization tasks, an open question on canonical benchmarks, not a universal claim. Our hope is that harnesses like Distillate augment research rather than automate it. If tomorrow researchers see further, perhaps it will be by standing on the shoulders of agents [Newton, 1675].

References

Ejas Althaf Abtheen, Arun Singh, Shyam Sriram, Changyou Chen, Sriram Neelamegham, and Rudiyanto Gunawan. Transformer-based deep learning for glycan structure inference from tandem mass spectrometry. *bioRxiv*, 2025. doi: 10.1101/2025.07.02.662857.

Anthropic. Model Context Protocol (MCP) specification. https://modelcontextprotocol.io, 2024.

P. Ball. *Alchemy: An Illustrated History of Elixirs, Experiments, and the Birth of Modern Science*. Yale University Press, 2025. ISBN 9780300280876.

J. Beel, M.-Y. Kan, and M. Baumgart. Evaluating sakana's AI Scientist: Bold claims, mixed results, and a promising future?, 2025.

D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023.

A. M. Bran, S. Cox, O. Schilter, C. Baldassari, A. D. White, and P. Schwaller. Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, 6(5):525–535, 2024. doi: 10.1038/s42256-024-00832-8.

J. S. Chan, N. Chowdhury, O. Jaffe, et al. MLE-bench: Evaluating machine learning agents on machine learning engineering, 2024. OpenAI.

Cognition AI. SWE-bench technical report (Devin). https://cognition.com/blog/swe-bench-technical-report, 2024.

X. Deng, J. Da, E. Pan, Y. Y. He, C. Ide, et al. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks?, 2025. Scale AI.

Edison Scientific. Kosmos: An AI scientist for autonomous discovery, November 2025.

A. Fourney, G. Bansal, H. Mozannar, C. Tan, E. Salinas, E. Zhu, et al. Magentic-One: A generalist multi-agent system for solving complex tasks, 2024. Microsoft Research.

S. Gao, A. Fang, Y. Huang, V. Giunchiglia, A. Noori, J. R. Schwarz, Y. Ektefaie, J. Kondic, and M. Zitnik. Empowering biomedical discovery with AI agents. *Cell*, 187(22):6125–6151, 2024.

J. M. Hofman, A. Chatzimparmpas, A. Sharma, D. J. Watts, and J. Hullman. Pre-registration for predictive modeling, 2023.

K. Huang et al. POPPER: Automated hypothesis validation with agentic sequential falsifications, 2025.

N. Huntington-Klein, C. C. Pörtner, I. McCarthy, and the Many Economists Collaborative on Researcher Variation. The sources of researcher variation in economics. Technical Report Working Paper No. 33729, NBER, 2025.

Z. Jiang et al. AIDE: AI-driven exploration in the space of code, 2025.

C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*, 2024.

S. Kadavath, T. Conerly, A. Askell, T. Henighan, et al. Language models (mostly) know what they know, 2022.

S. Kapoor and A. Narayanan. Leakage and the reproducibility crisis in ML-based science. *Patterns*, 4(9):100804, 2023. doi: 10.1016/j.patter.2023.100804.

A. Karpathy. Open educational projects: `nanoGPT`, `makemore`, `micrograd`, and the `LLM101n` curriculum. https://github.com/karpathy, 2022–2025.

O. Khattab, A. Singhvi, P. Maheshwari, et al. DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In *ICLR*, 2024.

A. Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

R. Lacombe. GlycoGen: Glycan structure prediction with crystallizing flows, 2026. Preprint.

R. Lacombe, K. Wu, E. Dilworth, et al. ClimateX: Do LLMs accurately assess human expert confidence in climate statements? In *NeurIPS Workshop on Tackling Climate Change with Machine Learning*, 2023.

R. Lacombe, K. Wu, E. Dilworth, et al. Don't think twice! Over-reasoning impairs confidence calibration. In *ICML Workshop on Reliable and Responsible Foundation Models*, 2025.

D. Lakens. Improving your statistical inferences—chapter 13: Preregistration and transparency. Open educational textbook, https://lakens.github.io/statistical_inferences/, 2022.

N. Lee, K. Sreenivasan, J. D. Lee, K. Lee, and D. Papailiopoulos. Teaching arithmetic to small transformers, 2023.

R. Li, D. Patel, Q. Wang, and X. Du. MLR-Copilot: Autonomous machine learning research based on large language model agents, August 2024.

S. Lin, J. Hilton, and O. Evans. Teaching models to express their uncertainty in words. *Transactions on Machine Learning Research*, 2022.

X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, et al. AgentBench: Evaluating LLMs as agents. In *ICLR*, 2024.

Chris Lu, Cong Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha. The AI Scientist: Towards fully automated open-ended scientific discovery, 2024. Sakana AI.

A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, et al. Self-Refine: Iterative refinement with self-feedback. In *NeurIPS*, 2023.

G. Mialon, C. Fourrier, C. Swift, T. Wolf, Y. LeCun, and T. Scialom. GAIA: A benchmark for general AI assistants, 2023.

I. Newton. Letter to Robert Hooke, 1675. Dated 5 February 1675; published in H. W. Turnbull (ed.), *The Correspondence of Isaac Newton, Vol. 1*, Cambridge University Press, 1959, pp. 416–417.

A. Novikov, N. V u, M. Eisenberger, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery, 2025. Google DeepMind.

C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023.

D. Papailiopoulos and contributors. AdderBoard: A community leaderboard for the smallest transformer that can add two ten-digit numbers. https://github.com/anadim/AdderBoard, 2024.

J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *UIST*, 2023.

S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez. Gorilla: Large language model connected with massive APIs, 2023.

B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.

T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. In *NeurIPS*, 2023.

S. Schmidgall, Y. Su, Z. Wang, X. Sun, J. Wu, X. Yu, J. Liu, M. Moor, Z. Liu, and E. Barsoum. Agent Laboratory: Using LLM agents as research assistants, 2025.

H. Semmelrock et al. Reproducibility in machine-learning-based research: Overview, barriers, and drivers. *AI Magazine*, 46(2):e70002, 2025.

Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang. HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. In *NeurIPS*, 2023.

N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.

K. Swanson, W. Wu, N. L. Bulaong, J. E. Pak, and J. Zou. The Virtual Lab: AI agents design new SARS-CoV-2 nanobodies with experimental validation. *bioRxiv*, 2024. doi: 10.1101/2024.11.11.623004.

T. H. Trinh, Y. Wu, Q. V. Le, H. He, and T. Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625:476–482, 2024. doi: 10.1038/s41586-023-06747-5.

G. Wang, Y. Xie, Y. Jiang, et al. Voyager: An open-ended embodied agent with large language models, 2023.

H. Wijk, T. Lin, J. Becker, S. Jawhar, N. Parikh, et al. RE-Bench: Evaluating frontier AI R&D capabilities of language model agents against human experts, 2024. METR.

Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation, 2023.

Y. Yamada, R. T. Lange, et al. The AI Scientist-v2: Workshop-level automated scientific discovery via agentic tree search, 2025.

S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. Tree of Thoughts: Deliberate problem solving with large language models. In *NeurIPS*, 2023a.

S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023b.

A. L. Zhang, T. Kraska, and O. Khattab. Recursive Language Models, 2025.

A. L. Zhang, Z. Li, and O. Khattab. The mismanaged geniuses hypothesis. Blog post, https://alexzhang13.github.io/blog/2026/mgh/, April 2026.

S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, et al. WebArena: A realistic web environment for building autonomous agents. In *ICLR*, 2024.

Responsible use

Autonomous ML research systems compress the cost of running experiments, which cuts both ways: harnesses like Distillate lower the barrier for rigorous pre-registered campaigns, but the same infrastructure could automate dual-use research or generate misleading artifacts at scale, and multi-day agent campaigns consume non-trivial energy that should be scoped and reported. Distillate embeds two mitigations we recommend as defaults: cryptographic pre-registration makes agent reasoning auditable by any third-party reader walking the Git history; bidirectional human-in-the-loop steering gives researchers a low-friction path to halt or redirect an experiment as it unfolds. Sandboxed enforcement, formal governance of what agents may research autonomously, and honest per-campaign compute reporting belong to the broader community response.

A The `start_run()` pre-registration schema

The seven-field schema referenced in Section 3. All fields are required. The tool returns `(run_id, run_number)` and writes the entry to `runs.jsonl` before training begins.

Field	Type	Description
`description`	string	The approach being tried this run
`hypothesis`	string	Why it should work
`prediction`	string	Concrete, falsifiable outcome
`predicted_metric`	string	Name of the primary metric
`predicted_value`	float	Numeric prediction
`confidence`	int	Calibration score, 0–100
`rationale`	string	Evidence (prior runs or papers)

B System prompt excerpts

Both agents' full system prompts ship with the released harness and are unit-test-snapshotted; excerpts below capture the load-bearing sections. Placeholders in `{{double-braces}}` are interpolated at runtime.

Orchestrator (Nicolas).

> *You are **Nicolas**, a research alchemist — named after Nicolas Flamel. Do NOT identify as Claude Code or Claude. Your name is Nicolas. You are the command and control center for a researcher's experimental work. . . .*
>
> **Lab REPL — Your Primary Analysis Tool.** You have a `mcp__distillate__lab_repl` tool: a persistent Python sandbox for multi-step reasoning. USE IT whenever a question spans multiple papers AND experiments, requires filtering / ranking / comparison, needs multi-step analysis, or would otherwise require 3+ sequential tool calls. Available: `lab.papers.{search, get, recent, queue, stats, highlights}`, `lab.experiments.{list, get, runs, active}`, `lab.notebook.{recent, digest}`, `llm_query(...)` (fast Haiku sub-call), `delegate(...)` (recursive sub-agent with data), `FINAL(answer)`. Always provide a human-readable `description`. For quick single lookups, use specific MCP tools directly.
>
> *Reflex: search before admitting ignorance. Before saying "I don't know" about any named model / paper / dataset / method, first call `lab.papers.search("<name>")` and `lab.experiments.active()`; only ask the user to fill in specs after the library search comes up empty.*

Experimentalist.

> *You are an **Experimentalist** — an autonomous research agent of the Distillate Lab, instantiated to advance a specific research goal. Read `PROMPT.md` and follow it precisely.*
>
> You are fully autonomous. The human may be asleep. Do not pause, do not ask questions, do not wait for input. Work indefinitely until manually stopped. If you are stuck, try a different approach.
>
> **Pause protocol.** If `start_run` returns `{"paused": true}`, the user has paused this experiment to ponder steering. Do NOT call `start_run` again — exit the session cleanly. The user will remove

`.distillate/pause_requested` (often after writing fresh `.distillate/steering.md`) and resume you in a new session.

Run Protocol. For EVERY run: *Step 0 Plan.* Read `runs.jsonl` and `context.md`; build on what worked, avoid repeating failures. Pre-register by calling `start_run(...)` (schema in Appendix A). The prediction must be concrete and falsifiable. *Step 1 Train.* Launch training through `distillate-run` (reads `.distillate/budget.json`, sends SIGTERM at the budget with a grace window). Never hardcode `MAX_SECONDS`; read it via `distillate.budget.read_train_budget()`. Print metrics incrementally. *Step 2 Conclude.* Call `conclude_run(run_id, actual_value, verdict, belief_update)`; the tool emits the merge commit and pushes.

C The .distillate filesystem layout

Each Distillate experiment is its own Git repository. A reserved `.distillate/` directory at the root holds all agent state:

```
my-experiment/
|-- PROMPT.md                # Orchestrator's brief (Question /
|                            #   Context / Optimization / Budget)
|-- train.py                 # training script
|-- .distillate/
|   |-- runs.jsonl           # append-only, prereg + conclude rows
|   |-- events.jsonl         # per-epoch metric stream from training
|   |-- context.md           # rolling Reflexion synthesis
|   |-- scratchpad.md        # experimentalist working memory
|   |-- budget.json          # train + wrap deadlines (per-run)
|   |-- steering.md          # researcher / Orchestrator hints
|   |-- pause_requested      # sentinel; refuses next start_run
|   |-- experiment.json      # metric_name, draft flag, overrides
|   |-- calibration.jsonl    # (confidence, verdict) probe pairs
|   |-- cost.json            # sub-LM ledger {model: total_usd}
|   `-- checkpoints/<run_id>/ # artefact cache (SHA-256 in conclude)
`-- .git/                    # code AND .distillate/ are tracked
```

Two commits per run. Each run produces two cryptographically linked Git commits. `start_run` fast-forward-rebases onto `origin/HEAD`, commits a `prereg:` entry staging the new `runs.jsonl` row and `budget.json`, and *pushes to origin before training launches*, anchoring the prediction in GitHub's receive-time and the content-addressed Git SHA. `conclude_run` merges `origin/HEAD` into the index without committing, then emits a `run:` merge commit whose parents are the prereg and the orchestrator's contemporary state. Trailers (`Distillate-Kind`, `Distillate-Run`, `Distillate-Verdict`, `Distillate-Metric`) make the trace machine-filterable. Two distinct authors (`Nicolas <nicolas@distillate.dev>` and `Experimentalist <experimentalist@distillate.dev>`) separate orchestration from execution in `git shortlog`.